

Reviewer Pack — Minimal Rank of Quadratic Forms over SAT Instances Bounds Re...

Entry 60871d321cb3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 15:55:13 UTC

Minimal Rank of Quadratic Forms over SAT Instances Bounds Resolution Refutation Size

Entry ID: 60871d321cb3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 15:55:13 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Quadratic Forms) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

{'property': 'ResolutionRefutationSize', 'relation': ' $\leq$ ', 'quantifier': ' Θ ', 'function': 'MinimalRankOfQuadraticForms', 'on': 'SATInstances'}

Rationale (proposer's reasoning):

['Quadratic forms are known to be expressive in representing solutions to polynomial equations. Their ranks can reveal the complexity of solving these equations.', 'Resolution proof complexity measures the length of a resolution proof, which is an efficient way to determine whether a given CNF formula is satisfiable or not.', 'A connection between quadratic form ranks and resolution refutation size could provide insights into both algebraic geometry and computational complexity.']

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9ea12d32a5518f27

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the minimal rank of quadratic forms and the resolution refutation size is ≤ -0.7 for at least 80% of the seeds, and no seed shows a positive correlation coefficient > 0.3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank of quadratic forms" AND "resolution proof complexity"
- "quadratic forms in algebraic geometry" AND "resolution refutation size" - "algebraic geometry" AND "complexity theory resolution"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.6s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            pivot_row = max(range(i, n), key=lambda k: abs(A[k][i]))
            A[i], A[pivot_row] = A[pivot_row], A[i]
            pivot = A[i][i]
            if pivot == 0:
                return float('inf') # Indeterminate rank
            for j in range(n):
                A[i][j] /= pivot
            for k in range(n):
                if k != i:
                    factor = -A[k][i]
                    for j in range(n):
                        A[k][j] += factor * A[i][j]
        return sum(1 for row in A if any(row))

    def minimal_rank_of_quadratic_form(clauses):
        n = len(clauses)
        Q = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i, n):
                Q[i][j] = sum(1 for clause in clauses if (i + 1) in clause and (j + 1) in clause)
                Q[j][i] = Q[i][j]
        return gaussian_elimination(Q)
```

```

def generate_sat_instance(n):
    variables = list(range(1, n + 1))
    clauses = []
    for _ in range(n):
        clause = random.sample(variables, random.randint(1, n))
        clauses.append(clause)
    return clauses

resolution_refutation_sizes = [random.randint(50, 200) for _ in range(30)]
ranks = []

for n in [5, 10, 15, 20, 30, 40]:
    instances_tested = 0
    for refutation_size in resolution_refutation_sizes:
        clauses = generate_sat_instance(n)
        rank = minimal_rank_of_quadratic_form(clauses)
        if rank != float('inf'):
            ranks.append(rank)
            instances_tested += 1

mean_rank = sum(ranks) / len(ranks)
std_rank = math.sqrt(sum((x - mean_rank) ** 2 for x in ranks) / len(ranks))

correlation_coefficient = sum((ranks[i] - mean_rank) * (resolution_refutation_sizes[i] - mean_rank)) / (len(ranks) * (std_rank * std_resolution_refutation_sizes))

conjecture_holds = correlation_coefficient <= -0.7
counterexample = "" if conjecture_holds else "correlation_coefficient > -0.7"

return {
    "metric_name": "MinimalRankOfQuadraticForms",
    "metric_value": mean_rank,
    "instances_tested": len(ranks),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif any(r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)

```


#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11838
9	judge	ollama_remote	glm4:latest	0	0	8247

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 85990 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/60871d321cb3.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/60871d321cb3.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/60871d321cb3.tar.gz (if generated)